

Tutorial3: Object-oriented programming in OCaml

- 应该尽量学会更多定义完备的模块函数，如 `List.iter`, `List.map` 之类。

1 类型进阶

1.1 常见类型与调用方法

- 元组 `a * b` 和 `(a, b)`，前者是定义元组的方式，后者是调用元组的方式。
访问：你可以使用模式匹配来访问元组中的元素。
- 记录 `{a:a1; b:b1}` 和 `{a = a1; b = b1}`，前者是定义记录的方式，后者是调用记录的方式。
访问：记录字段通过字段名来访问（使用点操作符 `.`）。

```
type person = { name: string; age: int };;
let person1 = { name = "Alice"; age = 30 };; (* 单独这一行代码会报错，要先声明*)
let name = person1.name;;
```

- 列表 `[a; b]`。访问：你可以通过递归模式匹配来访问列表中的元素，或者使用 `List.hd` 和 `List.tl` 来访问列表的头部和尾部。

1.2 大小写总结

类别	示例	规则说明
模块名	<code>String, List</code>	语法强制要求首字母大写（否则编译错误）
异常构造器	<code>Failure, Invalid_argument</code>	异常构造器必须首字母大写（语法强制）
变体构造器	<code>Red, Empty</code>	所有变体构造器必须首字母大写（语法强制）
多态变体标签	<code>`Success</code>	多态变体标签用反引号包裹，标签名本身无需大写，但常遵循惯例（如 <code>`Ok`</code> ）

类别	示例	规则说明
记录字段名	<code>width, name</code>	记录字段必须小写开头（语法强制）
变量/函数名	<code>x, calculate_area</code>	普通变量和函数名必须小写或下划线开头（语法强制）
类型名	<code>int, string</code>	类型名通常小写（社区惯例，非语法强制，但 <code>int</code> 等内置类型强制小写）
方法名	<code>get, draw</code>	类方法名必须小写（语法强制）
类型参数	<code>'a, 'key</code>	类型参数必须小写（语法强制）
类名	<code>class counter, class point</code>	类名首字母小写

1.3 变体（Variants）

```
type t = C1 [of t1] | ... | Cn [of tn]
```

- `Ci` 这些属于构造器，后面可以带参数。

```
type linked = Empty | Cons of int * linked;;
let lst = Cons(1, Cons(2, Cons(3, Empty)));;
type 'a linked = Empty | Cons of 'a * 'a linked;;
```

- 递归类型和其多态。

1.4 多态变体标签

```
let zhl = `Beautiful "big beauty" ;;
let ldk = `Wholehearted (true , 100) ;;
let xym = `Beautiful "silly beauty";;

let func x =
  match x with
  | `Beautiful msg -> print_endline msg
  | `Wholehearted (x, y) ->
    if x = true then print_endline ("he devoted " ^
string_of_int y ^ "%, wholeheartedly")
    else print_endline ("bad guy!")
  | _ -> print_endline "wrong!";;

func zhl;;
```

```
func ldk;;
func xym;;
```

- 要理解多态变体标签，他可以扩充类型的严格限制，`let ldk = 'Wholehearted (true , 100 );;` 执行的类型 `val ldk : [> 'Wholehearted of bool * int ] = 'Wholehearted (true, 100)`。函数体执行后的结果是 `val func : [> 'Beautiful of string | 'Wholehearted of bool * int ] -> unit = <fun>`，如果没有_的异常类型，则是 `[< ... ]`。可以从中体会二者的区别！

2 Error Handling

```
exception DivideByZero of string;;

let division x y =
  if y = 0 then raise (DivideByZero "YOU are STUPID!")
  else x / y;;

let result =
  try
    division 11 0
  with
  | DivideByZero msg -> print_endline msg; 0;;
```

- 尽可能在引用的时候再进行错误判定，这样灵活性更强。

3 Module和Class

3.1 class 和 module 的区别

Module使用规则	Class使用规则
module type 和 sig-end 配合使用	class type和object配合使用
module 和 struct-end 配合使用， 但是可以直接module MyList = List用于重命名模块	class和object配合使用
用open和include可以类似继承，前者只是改变域名解析，后者展开全部模块代码，二者作用域也不相同	用inherit实现继承，一般为区分子类和父类，用inherit father as super表示区分

Module使用规则	Class使用规则
val定义变量，不允许mutable，允许ref	val定义变量，允许mutable，不推荐ref
val声明方法，let定义方法	method声明、定义方法
module名大写首字母	class名小写首字母
变量默认是公开的，可以通过md.xxx	变量默认是私有的，只有方法公开

3.2 module 模块案例

```

module type People = sig
  type t
  val age : int ref
  val make : string -> bool -> t
  val change_gender : t -> t
  val add_age : int -> unit
  val print_info : t -> unit
end;;

module People : People = struct
  (* 在记录定义中, 使用 mutable 关键字标记 male 字段为可变 *)
  type t = { name : string ; mutable male : bool }
  let age = ref 20
  (* 创建一个新的对象 *)
  let make str is_male = { name = str; male = is_male }
  (* 改变性别 *)
  let change_gender t =
    if t.male = false then t.male <- true
    else t.male <- false;
    t (* 返回修改后的 t *)
  (* 增加年龄 *)
  let add_age num =
    age := num + !age
  (* 打印信息 *)
  let print_info t =
    match t.male with
    | true -> Printf.printf "name : %s, age : %d, male : true\n"
t.name !age
    | false -> Printf.printf "name : %s, age : %d, male : false\n"
t.name !age
end;;

open People;;

(* 创建一个名为 "ldk" 且性别为 true 的人 *)
let t = make "ldk" true;;

```

```
(* 增加年龄 20 *)
let () = add_age 20;;
(* 改变性别 *)
let t = change_gender t;;
(* 打印个人信息 *)
let () = print_info t;;
```

- 上述例子展示了：
 - 接口（或签名）的实现、模块的实现；
 - `ref` 的使用；
 - 模块的调用方式；
 - `type` 的定义和使用，注意接口中有 `type` 的定义，所以只能通过构造函数创建 `t`，否则可以直接在外侧调用；
 - `open` 的使用。

3.3 `class` 模块案例

```
(* 定义 teacher 的类型接口 *)
class type teacher_interface =
  object
    val name : string
    val mutable age : int
    val mutable male : bool
    val love_to_say : string
    method add_age : int -> unit
    method change_gender : unit -> unit (* unit -> unit 需要显式
    将()作为参数 *)
    method print_info : unit (* unit属于无参方法 *)
    method shout : string -> unit
  end;;

(* 定义 people 类 *)
class people (name : string) (age : int) (male : bool) =
  object(self)
    val name = name
    val mutable age = age
    val mutable male = male
    method add_age num = age <- age + num
    method change_gender () =
      if male = true then male <- false
      else male <- true ;()
    method print_info =
      match male with
      | true -> Printf.printf "name: %s; age: %d; male: true" name
      age
```

```

        | false -> Printf.printf "name: %s; age: %d; male: false"
name age
end;;

(* 定义 teacher 类, 继承自 people *)
class teacher (name : string) (age : int) (male : bool)
(love_to_say : string) : teacher_interface =
  object(self)
    inherit people name age male as super
    val love_to_say = love_to_say

    method shout msg =
      print_endline (name ^ ": " ^ love_to_say ^ msg)
  end;;

let my_teacher = new teacher "ldk" 25 true "You so cai!";;
my_teacher#add_age 15;;
my_teacher#change_gender ();;
my_teacher#shout "Stand up!";;
my_teacher#print_info;;

```

- 上述例子展示了：
 - 类的接口（或签名）、类的实现；
 - 类的继承关系及调用；
 - `unit` 为参数的函数、无参函数；
 - 类的调用方式。

```

class virtual animal =
  object
    val virtual name : string
    method virtual speak : string -> unit
    method laugh = print_endline "haha"
  end;;

class dog (n : string) =
  object
    inherit animal as super
    val name = n
    val mutable disorder_pee = true
    method speak str = print_endline (name ^ ": " ^ str ^ "
wangwang!")
  end

class people (n : string) =
  object
    inherit animal as super
    val name = n
    val iron = true

```

```

    method speak str = print_endline (name ^ ": " ^ str ^ " hi!")
end;;

let list =
  let x = new dog "ldk" in
  let y = new people "zh1" in
  let z = new dog "xym" in
  [x;y;z];;
List.iter (fun x -> x#speak "hello") list;;

let me = new people "ldk";;
me#speak "wawa";;
me#laugh;;
me#iron;; (* 默认为私有字段，需要用getter setter函数显式配置，会报错 *)

```

- 上面例子展示了：
 - 虚拟类的使用。
 - AOP(面向切向编程)的实际应用，因为本例 `people` 和 `dog` 中没有不相同的方法，所以可以不用强转，默认是同一个类，否则需要强转如下：

```

let list =
  let x = (new dog "ldk" :> animal) in
  let y = (new people "zh1" :> animal) in
  let z = (new dog "xym" :> animal) in
  [x;y;z];;

```

```

let p =
  object
    val mutable x = 0
    method get_x = x
    method move d = x <- x + d
  end;;

```

- 上面例子展示了：直接对象（Immediate Object），不使用class显式声明，直接用object和end进行包裹，可以放置在代码当中的任意位置。值得注意的是相比于完整的对象声明方法，直接对象不能被继承。

```

class ['a] stack =
  object (self)
    val mutable list = ([] : 'a list) (* instance variable *)
    method push x = (* push method *)
      list <- x :: list
    method pop = (* pop method *)

```

```

    let result = List.hd list in
    list <- List.tl list;
    result
  method peek = (* peek method *)
    List.hd list
  method size = (* size method *)
    List.length list
end;;
let s : int stack = new stack;;
let p = new stack;;
let () = p#push "ldk";;

```

- 上面例子展示了：类的多态，以及两种调用方法。

```

class my_class (num : int)=
  object (self)
    val mutable age = num
    method add_age num = age <- age + num
    method copy = Oo.copy self
    method print_info = Printf.printf "age: %d\n" age
  end ;;

let x = new my_class 20;;
let y = x#copy;;
let () = y#add_age 30;;
let () = x#print_info;;
let () = y#print_info;;

```

- Oo Module：如克隆对象 Oo.copy：该函数能生成一个对象的浅拷贝，method copy = Oo.copy self。还有标识对象 Oo.id：可以求取一个实例的标识符，同一个类的不同实例具有不同的标识符。利用这个函数可以便捷的判定两个实例是不是同一个实例（尽管二者可能是同一个类的实例）。

4 Functors（函子）

```

module type R_store = sig
  val r : float
end;;

module My_try = struct
  let r = 3.0
end;;

module Trans (M : R_store) = struct
  let pai = 3.14
end;;

```

```

let area = pai *. M.r *. M.r
let half_area = 0.5 *. pai *. M.r *. M.r
end;;

module Result = Trans(My_try);;

let rst = Result.half_area;;

```

- 理解为可以将一组数据，通过 `Trans` 转化为另一组数据，中间过程可以调用多个函数操作。

5 实验题

5.1 Calculator

题目描述

请使用 OCaml 语言，实现一个简单的算术表达式计算器。

我们使用以下代数数据类型 `expr` 来表示算术表达式，你需要补充乘法和除法的定义：

```

type expr =
| Const of int          (* 整数常量 *)
| Add of expr * expr    (* 加法运算 *)
| Sub of expr * expr    (* 减法运算 *)

```

请你编写一个函数 `evaluate`，用于计算给定表达式 `expr` 的值。函数 `evaluate` 应当满足：

```

val evaluate : expr -> int option

```

该函数返回 `Some v`，表示计算出的整数结果 `v`。该函数返回 `None`，表示计算失败，比如除以零的情况。

```

type expr =
| Const of int          (* 整数常量 *)
| Add of expr * expr    (* 加法运算 *)
| Sub of expr * expr    (* 减法运算 *)
| Mul of expr * expr    (* 乘法运算 *)
| Div of expr * expr    (* 除法运算 *)

(* 计算表达式的值 *)
let rec evaluate = function
| Const n -> Some n
| Add (e1, e2) -> (
    match evaluate e1, evaluate e2 with
    | Some v1, Some v2 -> Some (v1 + v2)
    | _ -> None
  )
| Sub (e1, e2) -> (
    match evaluate e1, evaluate e2 with
    | Some v1, Some v2 -> Some (v1 - v2)

```

```

    | _ -> None
  )
| Mul (e1, e2) -> (
  match evaluate e1, evaluate e2 with
  | Some v1, Some v2 -> Some (v1 * v2)
  | _ -> None
)
| Div (e1, e2) -> (
  match evaluate e1, evaluate e2 with
  | Some v1, Some 0 -> None (* 除零错误 *)
  | Some v1, Some v2 -> Some (v1 / v2)
  | _ -> None
)

(* 测试示例 *)
let expr = Add (Const 3, Mul (Const 2, Const 5)) (* 3 + 2 * 5 =
13 *)
let result = evaluate expr (* 应返回 Some 13 *)

let expr_div_zero = Div (Const 10, Const 0) (* 10 / 0, 应返回
None *)
let result_div_zero = evaluate expr_div_zero

```

5.2 Bank Account

题目描述

创建一个银行账户系统，使其具备以下功能：

- 账户创建：每个账户必须包含唯一的用户 ID 和所有者姓名。
- 存款：向账户余额中添加一定金额。如果存款金额为负数，抛出 `InvalidDeposit` 异常。
- 取款：从账户余额中取出一定金额：如果余额足够，正常扣除金额并返回。如果余额不足，抛出 `InsufficientFunds` 异常。如果取款金额为负数，抛出 `InvalidWithdrawal` 异常。
- 记录流水：你应该用一个 list 记录下来存取款的历史记录
- 查询余额：获取当前账户余额。

你需要实现一个 `bank_account` 类型。某些值应存储在一个可变字段或引用中，以便更新值。

```

type bank_account = {
  ...
}

```

你还需要实现四个函数：存款 `deposit`、取款 `withdraw`、查询余额 `get_balance` 以及输出记录 `print_transactions`

```

val deposit : bank_account -> int -> unit
val withdraw : bank_account -> int -> unit
val get_balance : bank_account -> int
val print_transactions : bank_account -> unit

```

```

type bank_account = {
  id: int;

```

```

    name : string;
    mutable balance : float;
    mutable transactions : (bool * float) list;
};;

exception InvalidDeposit of string;;
let deposit account (input_money : float) =
  try
    if input_money < 0.0 then raise (InvalidDeposit "Invalid
input!")
    else
      account.balance <- account.balance +. input_money;
      account.transactions <- (true, input_money) ::
account.transactions
  with
    | InvalidDeposit msg -> print_endline msg;;

exception InvalidWithdrawal of string;;
let withdraw account (output_money : float) =
  try
    if output_money > account.balance then raise
(InvalidWithdrawal "Out of money!")
    else if output_money < 0.0 then raise (InvalidWithdrawal
"Invalid output!")
    else
      account.balance <- account.balance -. output_money;
      account.transactions <- (false, output_money) ::
account.transactions
  with
    | InvalidWithdrawal msg -> print_endline msg;;

let get_balance account =
  account.balance;;

let print_function tuple =
  if fst tuple == true then print_endline ("deposit " ^
(string_of_float (snd tuple)))
  else print_endline ("withdraw " ^ (string_of_float (snd
tuple))));;
let print_transaction account =
  List.iter print_function account.transactions;;

let account = { id = 1; name = "Alice"; balance = 0.0;
transactions = [] };;

```

```
deposit account 100.0;;
withdraw account 50.0;;
get_balance account;;
print_transaction account;;
```

5.3 Map

题目描述

映射 (Map) 是由键值对组成的无序集合，在这个练习中你需要实现一个自己的 Map:

```
module type Map = sig
  (** [('k, 'v) t] 是一个映射的类型，它将键 ('k 类型) 绑定到值 ('v 类型)。 *)
  type ('k, 'v) t

  (** [empty] 表示一个不包含任何键的映射。 *)
  val empty : ('k, 'v) t

  (** [insert k v m] 返回一个新的映射，其中 [k] 绑定到 [v]，并且包含 [m] 中的所有绑定。
    如果 [k] 在 [m] 中已经有绑定，则新映射中 [k] 的绑定将被 [v] 替代。 *)
  val insert : 'k -> 'v -> ('k, 'v) t -> ('k, 'v) t

  (** [lookup k m] 返回在映射 [m] 中与键 [k] 绑定的值。
    如果 [k] 在 [m] 中没有绑定，则会引发 [Not_found] 异常。 *)
  val lookup : 'k -> ('k, 'v) t -> 'v

  (** [bindings m] 返回一个关联列表，该列表包含 [m] 中所有的绑定关系。
    该列表中的键保证是唯一的。 *)
  val bindings : ('k, 'v) t -> ('k * 'v) list
end
```

```
module type Map = sig
  (** [('k, 'v) t] 是一个映射的类型，它将键 ('k 类型) 绑定到值 ('v 类型)。 *)
  type ('k, 'v) t
  (** [empty] 表示一个不包含任何键的映射。 *)
  val empty : ('k, 'v) t
  (** [insert k v m] 返回一个新的映射，其中 [k] 绑定到 [v]，并且包含 [m] 中的所有绑定。
    如果 [k] 在 [m] 中已经有绑定，则新映射中 [k] 的绑定将被 [v] 替代。 *)
  val insert : 'k -> 'v -> ('k, 'v) t -> ('k, 'v) t
  (** [lookup k m] 返回在映射 [m] 中与键 [k] 绑定的值。
    如果 [k] 在 [m] 中没有绑定，则会引发 [Not_found] 异常。 *)
  val lookup : 'k -> ('k, 'v) t -> 'v
  (** [bindings m] 返回一个关联列表，该列表包含 [m] 中所有的绑定关系。
    该列表中的键保证是唯一的。 *)
  val bindings : ('k, 'v) t -> ('k * 'v) list
end;;

exception Not_found of string
module Map : Map = struct
  type ('k, 'v) t = ('k * 'v) list
end
```

```
let empty = []

let insert k v m =
  (k, v) :: m

let rec lookup k m =
  match m with
  | [] -> raise (Not_found "Key not found")
  | (key, value) :: t ->
    if key = k then value
    else lookup k t

let bindings m = m
end;;

let map = Map.empty;;
let map1 = Map.insert 1 "one" map;;
let map2 = Map.insert 2 "two" map1;;
let value = try
  Map.lookup 3 map
with
| Not_found msg -> Printf.printf "Error: %s\n" msg;
"default_value"
let all_bindings = Map.bindings map2;;
```

5.4 Pokemon Fight

题目描述

你需要模拟一个简单的宝可梦（Pokémon）战斗系统：

1. 定义 `Pokemon` 模块类型，其中包含属性：宝可梦的名称、生命值、攻击力、受到攻击时减少生命值：

```
module type Pokemon = sig
  (* 名称 *)
  val name : string
  (* 生命值 *)
  val hp : int ref
  (* 攻击力 *)
  val attack : int
  (* 收到攻击时减少生命值，减少的数量应该是对方宝可梦的攻击力 *)
  val take_damage : int -> unit
end
```

2. 实现具体的宝可梦模块，它们应该符合 `Pokemon` 模块类型，比如：

```
module Pikachu : Pokemon = struct
  let name = "Pikachu"
  ...
end
```

3. 定义 `Battle` Functor，它接受两个符合 `Pokemon` 模块类型的模块，并提供战斗逻辑：

- `fight : unit -> unit` — 两个宝可梦轮流攻击，直到其中一个生命值为 0。
- `print_winner : unit -> unit` — 输出胜利者的名字。

```
module Battle : functor (P1 : Pokemon) (P2 : Pokemon) -> sig
  val fight : unit -> unit
  val print_winner : unit -> unit
end
```

```
module type Pokemon = sig
  val name : string
  val hp : int ref
  val attack : int
  val take_damage : int -> unit
end;;

module Pikachu : Pokemon = struct
  let name = "Pikachu"
  let hp = ref 100
  let attack = 20
  let take_damage dam =
    if !hp >= dam then hp := !hp - dam
    else hp := 0
end;;

module Charmander : Pokemon = struct
  let name = "Charmander"
  let hp = ref 200
  let attack = 5
  let take_damage dam =
    if !hp >= dam then hp := !hp - dam
```

```

        else hp := 0
    end;;

module Battle (P1 : Pokemon) (P2 : Pokemon) = struct
    let fight () =
        while !P1.hp > 0 && !P2.hp > 0 do
            P2.take_damage P1.attack;
            P1.take_damage P2.attack;
        done
    let print_winner () =
        if !P1.hp > 0 then Printf.printf "%s wins!\n" P1.name
        else if !P2.hp > 0 then Printf.printf "%s wins!\n" P2.name
        else Printf.printf "It's a draw!\n"
    end;;

module PikachuVsCharmander = Battle(Pikachu)(Charmander);;

let () =
    PikachuVsCharmander.fight ();
    PikachuVsCharmander.print_winner ();;

```

5.5 Task

题目描述

你需要设计一个任务管理系统，支持管理不同类型的任务（普通任务、定时任务、优先任务），并实现以下功能：

1. 任务类型：
 - 普通任务：仅包含标题和描述。
 - 定时任务：在普通任务的基础上还包含开始时间和结束时间。
 - 优先任务：在普通任务的基础上还包含优先级（如 1-5，数字越大优先级越高）。
2. 任务管理器：开发一个任务管理器管理所有任务。
2. 任务操作：
 - 添加任务：向任务管理器添加任务。
 - 列出任务：以字符串格式显示所有任务信息。
 - 排序任务：先按优先级排序（优先任务 > 其他任务）。再按开始时间排序（仅适用于定时任务）。
 - 状态管理：任务可以标记为完成或未完成。

```

(* 任务类型 *)
type task_type =
    | Normal
    | Timed of { start_time: float; end_time: float }
    | Priority of { priority_level: int }

(* 任务状态 *)
type task_status =
    | Incomplete

```

| Complete

```
(* 基类任务 *)
class virtual task (title:string) (desc:string)
(task_type:task_type) = object (self)
  val title = title
  val desc = desc
  val task_type = task_type
  val mutable status = Incomplete

  method get_title = title
  method get_desc = desc
  method get_status = status
  method set_status s = status <- s
  method get_task_type = task_type
end

(* 定时任务 *)
class timed_task title desc start_time end_time = object
  inherit task title desc (Timed { start_time; end_time })
end

(* 优先任务 *)
class priority_task title desc priority_level = object
  inherit task title desc (Priority { priority_level })
end

(* 普通任务 *)
class normal_task title desc = object
  inherit task title desc Normal
end

(* 任务管理器 *)
class task_manager = object
  val mutable tasks : task list = []

  (* 添加任务 *)
  method add_task (t: task) = tasks <- t :: tasks

  (* 列出任务 *)
  method list_tasks = List.iter (fun t -> Printf.printf "Title:
%s, Desc: %s\n" (t#get_title) (t#get_desc)) tasks

  (* 排序任务, 优先级 > 时间 *)
  method sort_tasks =
    tasks <- List.sort (fun t1 t2 -> match (t1#get_task_type,
t2#get_task_type) with
      | (Priority {priority_level=p1}, Priority
{priority_level=p2}) -> compare p2 p1
```

```

        | (Timed {start_time=s1; _}, Timed {start_time=s2; _}) ->
compare s1 s2
        | _ -> 0) tasks

(* 完成任务 *)
method complete_task title =
    let task = List.find (fun t -> t#get_title = title) tasks in
    task#set_status Complete
end

let tm = new task_manager;;

let task1 = new normal_task "Buy milk" "Buy milk from the store";;
let task2 = new timed_task "Meeting" "Team meeting at 10am" 10.0
11.0;;
let task3 = new priority_task "Urgent Report" "Finish report" 5;;

tm#add_task task1;;
tm#add_task task2;;
tm#add_task task3;;

tm#list_tasks;;
tm#sort_tasks;;
tm#list_tasks;;

tm#complete_task "Buy milk";;

```